LRU Cache & Event Scheduler

Production-grade solutions — algorithms, analysis, and code

Naman Gautam
SE Intern Submission

March 16, 2026

Executive summary

This report contains production-quality solutions for two tasks:

1. An **O(1)** LRU cache implemented using `std::unordered_map` + `std::list`, with rationale and complexity analysis.
2. An event scheduler with functions to test attendability, compute minimum rooms required, and assign concrete room indices using a heap-based greedy algorithm.

Appendices include full, compilable C++ source files and a minimal test harness.

Contents

1 Problem statements (concise) **3**
- 1.1 LRU Cache . . . 3
- 1.2 Event Scheduler . . . 3

2 Design overview & justification **3**
- 2.1 LRU cache — data structures choice . . . 3
- 2.2 Scheduler — algorithmic pattern . . . 3

3 Algorithms (high-level pseudocode) **4**
- 3.1 LRU cache . . . 4
- 3.2 Scheduler (min rooms) . . . 4

4 Complexity summary **4**

5 Correctness sketch **4**
- 5.1 LRU . . . 4
- 5.2 Scheduler . . . 5

6 Practical notes and trade-offs **5**

7 Build, test and run **5**

8 Representative test cases **5**

Appendices **6**

Appendix A — LRU cache 6

Appendix B — Scheduler 7

Appendix C — Example main 9

1 Problem statements (concise)

1.1 LRU Cache

Implement a fixed-capacity cache that supports:

- `int get(int key)` — returns the value or `-1` if missing; marks the key most-recently-used.
- `void put(int key,int value)` — inserts or updates; evicts the least-recently-used item if capacity is exceeded.

Both operations must be amortized $O(1)$.

1.2 Event Scheduler

Given a list of intervals $(\text{start}, \text{end})$:

- `can_attend_all(events)`: return `true` if no overlapping intervals exist (adjacent allowed).
- `min_rooms_required(events)`: compute the minimum number of rooms required to host all events concurrently.
- `assignRooms(events)`: (optional) produce integer room assignments for each event such that no overlapping events share a room.

2 Design overview & justification

2.1 LRU cache — data structures choice

To achieve $O(1)$ for both operations:

- Use an `unordered_map<key, iterator>` for constant-time lookup from key to its node.
- Maintain recency with a doubly-linked list (`std::list<pair<key,value»`); front = most recent, back = least recent. Given an iterator, `splice` and removals are constant time.

This combination minimizes per-operation cost while keeping implementation straightforward and memory usage proportional to capacity.

2.2 Scheduler — algorithmic pattern

- `can_attend_all`: sort intervals by start time and check for any start $<$ previous end (strict overlap). Sorting yields $O(n \log n)$.
- `min_rooms_required`: sort by start times, sweep using a min-heap of end times: for each event, free all rooms whose end $\leq$ start, then reserve a room (push end). The maximum heap size observed is the required room count — optimal and runs in $O(n \log n)$.
- `assignRooms`: extend the above by maintaining room ids; reuse freed room ids via a small free-pool (min-heap) so produced ids are compact.

3 Algorithms (high-level pseudocode)

3.1 LRU cache

LRU operations (pseudocode)

get(key):

1. Lookup node = map[key]. If missing return -1.
2. Move node to front of list (most recent).
3. Return node.value.

put(key, value):

1. If key exists: update value, move node to front, return.
2. If size == capacity: evict node at back (remove from list and map).
3. Insert new node at front and add map[key] = iterator.

3.2 Scheduler (min rooms)

Interval partitioning (pseudocode)

1. Sort events by start time.
2. Initialize empty min-heap of end times.
3. For each event (s,e):
   - While heap not empty and heap.top() $\leq$ s: pop() (room freed).
   - push(e) (occupy a room until e).
   - track max heap size.
4. Return tracked max size.

4 Complexity summary

Function	Time complexity	Space complexity
LRU `get`	$O(1)$	$O(1)$ (per op)
LRU `put`	$O(1)$ amortized	$O(1)$ (per op)
Scheduler: sort step	$O(n \log n)$	$O(n)$
Scheduler: sweep/heap step	$O(n \log n)$ total	$O(n)$

Table 1: Asymptotic bounds; n = number of events.

5 Correctness sketch

5.1 LRU

The map ensures exact presence checks and constant-time access; moving nodes to the front on access/update preserves recency ordering. Eviction removes the list's back element, which by construction is the least recently used.

5.2 Scheduler

The heap-based greedy is the standard interval partitioning solution: whenever an event starts, any room with end $\leq$ start is reusable; if none exist, a new room is required. The algorithm produces the interval depth (maximum concurrency), which equals the minimum number of rooms necessary.

6 Practical notes and trade-offs

- **Memory vs speed:** the LRU approach uses per-entry pointers (list nodes) plus hash entries — modest overhead in exchange for $O(1)$ operations.
- **Alternatives:** language built-ins (Python's `OrderedDict`) or specialized caches (LRU-K, 2Q) can be used where workload characteristics demand them.
- **Concurrency:** for thread-safety, wrap public operations in a mutex (coarse-grained). For higher throughput, consider lock striping or established concurrent collections.

7 Build, test and run

Compile:

```
g++ -std=c++17 main.cpp -O2 -o assignment
```

Run:

```
./assignment
```

8 Representative test cases

- LRU: insert up to capacity, access older keys to change recency, insert new key to force eviction; assert evicted key is LRU.
- Scheduler:
  - Non-overlapping adjacent intervals: $(9, 10), (10, 11) \Rightarrow$ can attend all, rooms $= 1$.
  - Overlap: $(9, 11), (10, 12) \Rightarrow$ cannot attend all, rooms $= 2$.
  - Mixed: $(9, 10), (9, 11), (10, 11) \Rightarrow$ rooms $= 2$.

Appendices

Appendix A — C++: LRU cache

```cpp
#pragma once
#include <list>
#include <unordered_map>
#include <stdexcept>

class LRUCache {
public:
    LRUCache(size_t capacity) : capacity_(capacity) {
        if (capacity_ == 0)
            throw std::invalid_argument("Capacity must be greater than zero");
    }

    int get(int key) {
        auto it = map_.find(key);

        if (it == map_.end())
            return -1;

        cache_.splice(cache_.begin(), cache_, it->second);

        return it->second->second;
    }

    void put(int key, int value) {

        auto it = map_.find(key);

        if (it != map_.end()) {
            it->second->second = value;
            cache_.splice(cache_.begin(), cache_, it->second);
            return;
        }

        if (cache_.size() == capacity_) {

            auto last = cache_.back();
            map_.erase(last.first);
            cache_.pop_back();
        }

        cache_.push_front({key, value});
        map_[key] = cache_.begin();
    }

    void printCache() {

        for (auto &p : cache_) {
            std::cout << "(" << p.first << "," << p.second << ") ";
        }

        std::cout << std::endl;
    }

private:

    size_t capacity_;

    std::list<std::pair<int,int>> cache_;
```

```

    std::unordered_map<int, std::list<std::pair<int,int>>::iterator> map_;
};
```

Listing 1: lru_cache.hpp — unordered_map + std::list implementation

Appendix B — C++: Scheduler

```
#pragma once
#include <vector>
#include <algorithm>
#include <queue>
#include <tuple>

using Interval = std::pair<int,int>;

bool canAttendAll(std::vector<Interval> events)
{
    if(events.empty())
        return true;

    std::sort(events.begin(), events.end());

    int prev_end = events[0].second;

    for(size_t i=1;i<events.size();i++)
    {
        int start = events[i].first;

        if(start < prev_end)
            return false;

        prev_end = events[i].second;
    }

    return true;
}


int minRoomsRequired(std::vector<Interval> events)
{
    if(events.empty())
        return 0;

    std::sort(events.begin(), events.end());

    std::priority_queue<int,std::vector<int>,std::greater<int>> pq;

    int maxRooms = 0;

    for(auto &e : events)
    {
        int start = e.first;
        int end = e.second;

        while(!pq.empty() && pq.top() <= start)
            pq.pop();

        pq.push(end);

        maxRooms = std::max(maxRooms,(int)pq.size());
    }
```

```

    return maxRooms;
}


std::pair<int,std::vector<int>> assignRooms(std::vector<Interval> events)
{

    int n = events.size();

    std::vector<std::tuple<int,int,int>> ev;

    for(int i=0;i<n;i++)
        ev.push_back({events[i].first,events[i].second,i});

    std::sort(ev.begin(),ev.end());

    std::priority_queue<
        std::pair<int,int>,
        std::vector<std::pair<int,int>>,
        std::greater<std::pair<int,int>>
    > busy;

    std::priority_queue<int,std::vector<int>,std::greater<int>> freeRooms;

    std::vector<int> allocation(n);

    int nextRoom=0;

    for(auto &t:ev)
    {
        int start=std::get<0>(t);
        int end=std::get<1>(t);
        int index=std::get<2>(t);

        while(!busy.empty() && busy.top().first <= start)
        {
            freeRooms.push(busy.top().second);
            busy.pop();
        }

        int room;

        if(!freeRooms.empty())
        {
            room = freeRooms.top();
            freeRooms.pop();
        }
        else
        {
            room = nextRoom++;
        }

        allocation[index]=room;

        busy.push({end,room});
    }

    return {nextRoom,allocation};
}
```

Listing 2: scheduler.hpp — canAttendAll, minRoomsRequired, assignRooms

Appendix C — C++: Example main

```cpp
#include <iostream>
#include "lru_cache.hpp"
#include "scheduler.hpp"

int main()
{

    std::cout<<"------ LRU Cache Test ------"<<std::endl;

    LRUCache cache(3);

    cache.put(1,10);
    cache.put(2,20);
    cache.put(3,30);

    cache.printCache();

    std::cout<<"Get 1: "<<cache.get(1)<<std::endl;

    cache.put(4,40);

    cache.printCache();




    std::cout<<"\n------ Scheduler Test ------"<<std::endl;

    std::vector<Interval> events =
    {
        {9,10},
        {10,11},
        {11,12},
        {9,11}
    };

    if(canAttendAll(events))
        std::cout<<"Can attend all meetings"<<std::endl;
    else
        std::cout<<"Cannot attend all meetings"<<std::endl;



    std::cout<<"Minimum rooms required: "<<minRoomsRequired(events)<<std::endl;



    auto result = assignRooms(events);

    int rooms = result.first;
    std::vector<int> allocation = result.second;

    std::cout<<"Total rooms allocated: "<<rooms<<std::endl;

    for(int i=0;i<allocation.size();i++)
    {
        std::cout<<"Event "<<i<<" -> Room "<<allocation[i]<<std::endl;
    }

    return 0;
}
```

Listing 3: main.cpp — minimal test harness

References

1. E. O'Neil, P. O'Neil, G. Weikum. *The LRU-K Page Replacement Algorithm for Database Disk Buffering*, SIGMOD, 1993.
2. T. Johnson, D. Shasha. *2Q: A Low Overhead High Performance Buffer Management Replacement Algorithm*, VLDB, 1994.
3. J. Kleinberg, É. Tardos. *Algorithm Design*, Interval partitioning (lecture notes).
4. Recent surveys on learned cache replacement (arXiv/conference literature, 2021–2024).